Fase 2A.5 · Tipos de entidad (Maestro / Documento)

Resumen visual del plan · 2026-11-02 · decisiones D1–D12 (§22.23)

Objetivo: toda entidad de tabla declara **maestro** o **documento** y hereda del kernel: ciclo de vida y aprobaciones, contabilización automática, snapshots, plantillas PDF, e-mails de estado y Notificar, acceso por registro y por menú, Datos adicionales, importar/exportar, barra de botones estándar y menú de cuatro secciones.

17 tareas. Test primero; migraciones 0040–0046 hacia adelante.

Botones: Grabar · Eliminar · Nuevo · Imprimir · Anular · Aprobar · No Aprobar · Notificar

Reglas clave: el creador puede aprobar · contabilizar es obligatorio y atómico · los accesos solo restringen; el admin nunca.

Arquitectura por capas

Módulos (core · accounting · support)

Solo DECLARAN: entityType, typed.lines, typedHooks

Kernel · typed-entities (regla 4)

- **Transiciones** — tabla pura, sin I/O
- **Runner de ciclo** — 6 procesos genéricos
- **Contab. automática** — hook post, misma txn
- **Snapshots** — append-only
- **Plantillas PDF** — @react-pdf/renderer
- **E-mail + Notificar** — outbox → Mailer
- **Acceso** — por registro y por menú

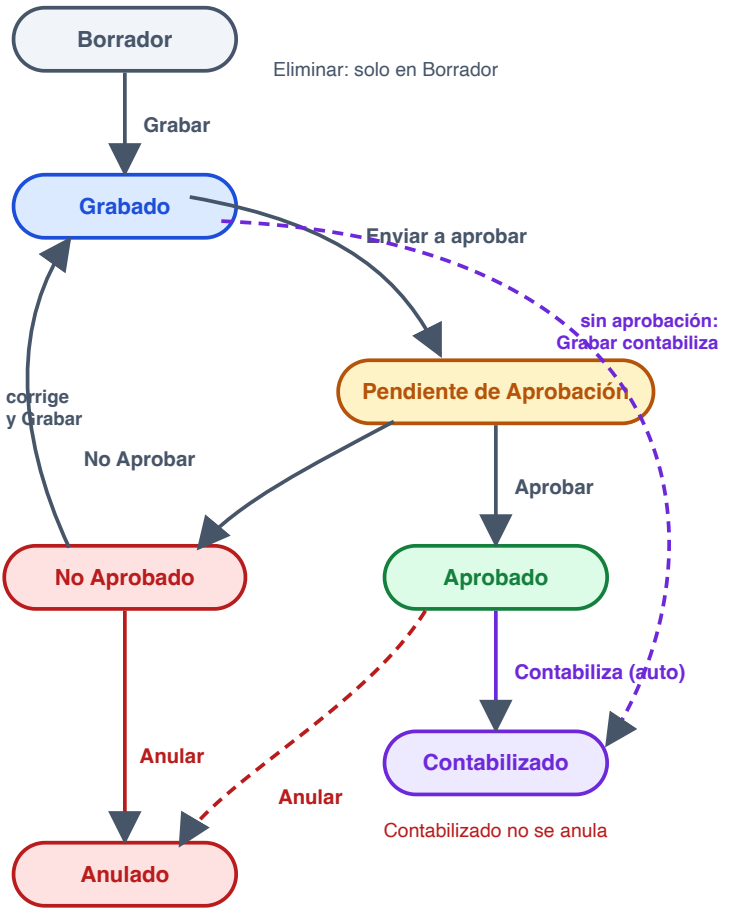
runProcess + RLS + auditoría

Todo cambio de estado es un proceso (regla 1)

Postgres · migraciones 0040–0046

settings · snapshots · record_access · menu_access · plantillas

Ciclo de vida del documento

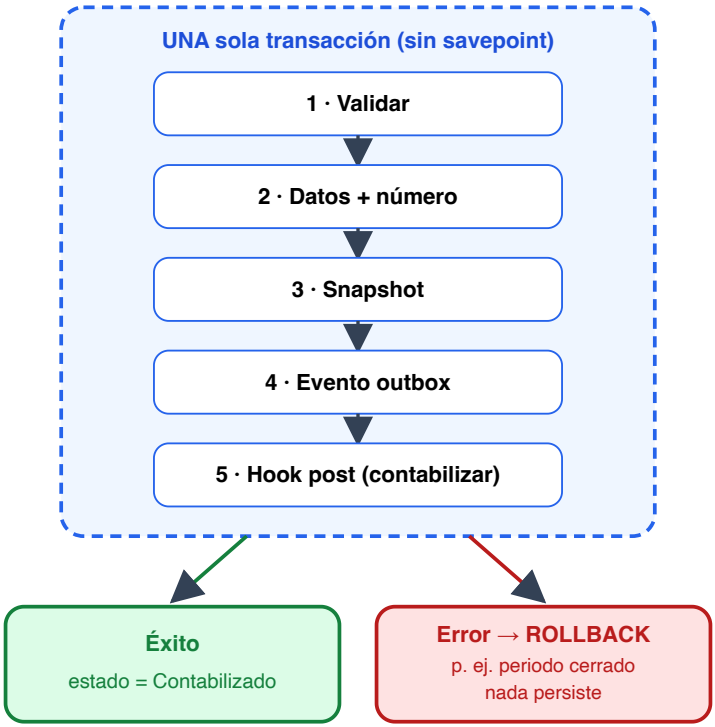


Maestro: sin Contabilizado/Anulado; Eliminar es borrado lógico (bloqueado si hay referencias); usa `is_active`; editar un campo de aprobación de un maestro aprobado lo devuelve a Pendiente (D10).

Usable: un maestro solo se usa en documentos si está Grabado (sin aprobación) o Aprobado, y activo.

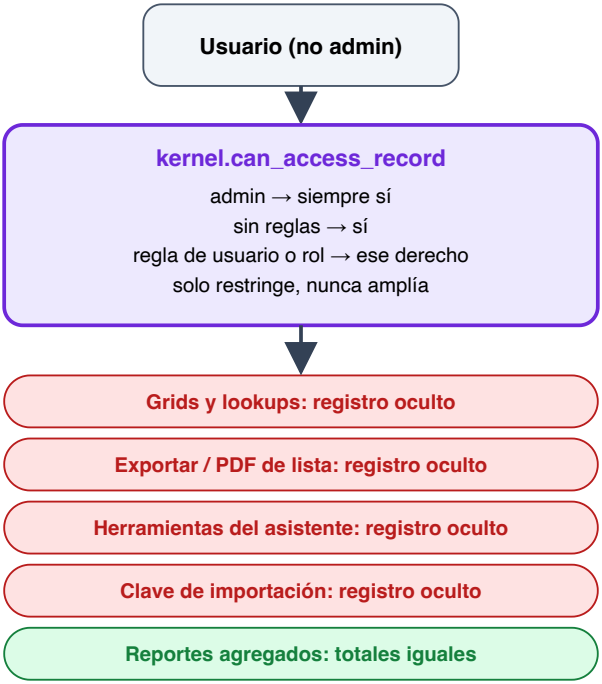
Edición: datos editables solo en Borrador, Grabado y No Aprobado.

Contabilización automática atómica



Grabar → sigue en Borrador · Aprobar → sigue en Pendiente

Acceso por registro



Maestro: leer · usar · modificar · eliminar
Documento: leer · modificar · anular

Menú en cuatro secciones



Una sección vacía muestra «Aún no hay elementos en esta sección». `kernel.menu_access` filtra el shell; el guard de ruta y `runProcess` rechazan con `forbidden`.

Enfoque de revisión

1. **Documento que no puede contabilizarse**
   Falla toda la acción; nada a medias. *(T6)*

2. **Maestro restringido**
   Invisible en grids, exportes, asistente e importación; totales agregados intactos. *(T9, T13)*

3. **Concurrencia**
   El segundo actor recibe stale_record. *(T5)*

4. **Cambiar flags en Parámetros**
   No altera registros existentes. *(T3)*

5. **Filas importadas**
   Nunca aprobadas ni contabilizadas; maestro con aprobación queda Pendiente. *(T13)*

Orden de ejecución

Paso	Tareas
1	T1
2	T2
3	T3
4	T4 · T5 (tras T3)
5	T6 · T11 · T14 (T11 y T14 tras T3)
6	T7 · T12 (tras T11)
7	T8
8	T9
9	T10 · T13 (tras T9)
10	T15 (tras T8, T9, T14)
11	T16 (tras T11, T12, T15)
12	T17 (último)

- Kernel
- Migración / BD
- UI / prototipo
- Docs y cierre

Las 17 tareas

1	**Declaración entityType + check-rules** entityType obligatorio
2	**Tabla de transiciones** nextStatus, allowedActions
3	**Tablas kernel (0040)** settings, snapshots, recipients
4	**Ajustes y destinatarios** 2 procesos admin

5	**Runner + snapshots** runLifecycle, 6 procesos
6	**Contabilización automática** post en la misma transacción
7	**Maestro usable** record_usable
8	**Migrar core y asientos** 0042/0043; return_to_draft → reject
9	**Acceso por registro** record_access
10	**Menú por secciones** 4 secciones, menu_access
11	**Plantillas PDF** bloques, variables, defaults
12	**E-mails de estado** Notificar, Resend
13	**Importar/exportar** nunca aprueba ni contabiliza
14	**Prototipo HTML** aprobación del dueño antes de T15
15	**Barra de acciones** RecordActions, Versiones, Acceso
16	**Pantallas Parámetros** editor de plantillas, imprimir lista
17	**Asistente y cierre** evals, guías, compuerta completa

Texto completo del plan

Phase 2A.5 · Entity types (Maestro / Documento) Implementation Plan

> **For agentic workers:** REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (`- [ ]`) syntax for tracking. Read `docs/project-rules.md` and the design sections for each task before starting; follow the plan task verbatim.

Goal: Every table entity declares `maestro` or `documento` and inherits from the kernel the lifecycle and approvals, automatic posting, snapshots, PDF templates, state e-mails and Notificar, record-level and menu access, Datos adicionales, import/export, the standard button bar and its place in the four-section module menu.

Architecture: A kernel `typed-entities` layer (module-agnostic, rule 4): declarations on `EntityDef`, a pure transition table, a lifecycle runner used by generic processes registered per typed entity, kernel tables for settings, snapshots, templates, recipients, record and menu access, all through `runProcess`, RLS and the outbox. Modules only declare; core masters and accounting entries are migrated onto it last.

Tech Stack: pnpm monorepo, Supabase Postgres (forward-only SQL migrations), TypeScript kernel, `@react-pdf/renderer`, `Mailer` seam, Next.js 16 App Router, shadcn + Tailwind v4, Vitest + Playwright.

Spec: `docs/superpowers/specs/2026-11-02-entity-types-design.md` (owner decisions D1–D12, 2026-11-02; main design §22.23).

Global Constraints

- Every state change is a registered process run through `runProcess` (rule 1); cross-module only through published views and outbox events (rule 3); kernel never names a module (rule 4).
- New tables: `id uuid` v7, `company_id`, `created_at/by`, `updated_at/by`, `seq` from `core.next_sequence`, `extra`/`extra_free` where business data (rule 7); RLS via `kernel.enable_company_rls()`; row audit via `kernel.enable_row_audit()`.
- Migrations forward-only, `NNNN_<module>_<topic>.sql`, starting at `0040` (rule 34); create with `pnpm db:migrate:new`.

- State codes exactly: `draft, saved, pending, rejected, approved, posted, voided`; labels es-419: Borrador, Grabado, Pendiente de Aprobación, No Aprobado, Aprobado, Contabilizado, Anulado.
- Button labels exactly: Grabar, Eliminar, Nuevo, Imprimir, Anular, Aprobar, No Aprobar, Notificar. Menu sections exactly: Maestros, Documentos, Reportes, Parámetros. All copy through `packages/i18n` (es-419 and en).
- Money `$ 1.234.567,00`. Snapshots and access-rule history are append-only.
- Access rules only narrow, never widen; company admins are never narrowed (D7, D12). Hiding is never the only guard.
- The creator or last editor may approve. Contabiliza → automatic posting (D8); posting is mandatory and part of the action's transaction: if it cannot post, the whole action rolls back and the document keeps its previous state.
- Eliminar on a documento only in `draft` (rule 9 as amended); maestro Eliminar is a soft delete.
- No commit or push unless the owner asks; the "Commit" steps below are staging points, run only on the owner's word.
- A feature without its user guide (`docs/user-guide/es-419/`) is not finished.

Review Focus

1. A document that cannot be posted (closed period, unbalanced, non-ready party) makes the whole action fail: Grabar leaves it `draft` with nothing persisted, Aprobar leaves it `pending`; no half-written entry, no consumed number, no snapshot, no e-mail — Task 6.
2. A user narrowed out of a maestro still sees totals in aggregate reports but not the record in any grid, lookup, export, PDF list, assistant tool or import key match — Tasks 9, 13.
3. Two users acting on the same document at once (Grabar while another approves) — the second gets `stale_record`, not a lost update — Task 5.
4. Changing `requiere_aprobacion` / `contabiliza` in Parámetros does not alter existing records (flags copied at create) — Task 3.
5. Imported rows go through the same lifecycle: an imported documento never lands posted or approved, an imported maestro needing approval lands `pending` — Task 13.

Task 1: Entity type declaration and rule check

Files:

- Modify: `packages/kernel/src/registry/entities.ts`, `packages/kernel/src/registry/types.ts`
- Modify: `scripts/check-rules.ts`
- Test: `packages/kernel/src/__tests__/registry.test.ts`, `scripts/__tests__/check-rules.test.ts`

Interfaces:

- Produces: `EntityDef.entityType?: "maestro" | "documento" | "lines" | "parametro" | "sistema"` (required when `kind !== "query"`, enforced by `defineEntity`; only maestro/documento are "typed" — D13, owner 2026-11-02); `EntityDef.typed?: { approvalFields?: string[]; lines?: { entity: string; parentField: string }; numbering?: "on_save" | "on_post" | "none" }`; `ResolvedEntityDef.entityType`.
- Produces: `checkEntityTypes(root): Violation[]` in check-rules — fails a table entity without `entityType`, a typed entity (maestro/documento) without `extra`/`extra_free` fields or without export enabled, a maestro without `importKey`, and a documento without `typed.lines` header+lines import declaration (D14); a `lines` entity must be named by some parent's `typed.lines`.
- **Step 1: Failing tests** — `defineEntity({...kind:"table"})` without `entityType` throws `entity core.x: entityType is required`; `kind:"query"` with `entityType` throws `query entities carry no entityType`; `approvalFields` naming a non-field throws. check-rules fixture without type reports `rule 7/§22.23: core.x has no entityType`.
- **Step 2: Run** `pnpm vitest run packages/kernel/src/__tests__/registry.test.ts scripts/__tests__/check-rules.test.ts` → FAIL.
- **Step 3: Implement** the fields and validation; add `entityType` to every existing table entity per design §12 (core parties, products, contacts, categories, accounting masters → `maestro`; `accounting.entry` → `documento`, `numbering: "on_post"`). Settings tables are not entities.
- **Step 4: Run** the tests and `pnpm check:rules` → PASS.
- **Step 5: Commit** `feat(kernel): entity types maestro and documento`.

Task 2: Pure lifecycle transition table

Files:

- Create: `packages/kernel/src/typed/lifecycle.ts`, `packages/kernel/src/typed/index.ts`

- Test: `packages/kernel/src/typed/__tests__/lifecycle.test.ts`

Interfaces:

- Produces: `type RecordStatus = "draft"|"saved"|"pending"|"rejected"|"approved"|"posted"|"voided"`; `type LifecycleAction = "save"|"submit"|"approve"|"reject"|"void"|"delete"|"post"`; `interface LifecycleFlags { requiresApproval: boolean; posts: boolean }`; `nextStatus(type: "maestro"|"documento", from: RecordStatus, action: LifecycleAction, flags: LifecycleFlags): RecordStatus | "deleted" | null` (null = refused); `canEditData(type, status): boolean`; `allowedActions(type, status, flags): LifecycleAction[]`; `isUsable(status, isActive, flags): boolean`.

- **Step 1: Failing table-driven tests** pinning design §5.2–5.3: documento `draft+save→saved`; `saved+submit→pending` only if requiresApproval; `pending+approve→approved`, `+reject→rejected`; `rejected+save→saved`; `saved|pending|rejected|approved + void → voided`; `posted+void → null`; `draft+delete→"deleted"`, `saved+delete→null`; `post` only from `saved` (no approval) or `approved`, only when `posts`; documento `canEditData` true only for draft/saved/rejected; maestro has no `posted`/`voided`, `delete` allowed from any state, `canEditData` always true; `isUsable("saved", true, {requiresApproval:false})` true, `isUsable("saved", true, {requiresApproval:true})` false, `isUsable("approved", false, …)` false.

- **Step 2: Run** `pnpm vitest run packages/kernel/src/typed/__tests__/lifecycle.test.ts` → FAIL.
- **Step 3: Implement** as a const transition map; no I/O.
- **Step 4: Run** → PASS. **Step 5: Commit** `feat(kernel): typed entity lifecycle table`.

Task 3: Kernel tables for typed entities

Files:

- Create: `supabase/migrations/0040_kernel_typed_entities.sql`
- Test: `packages/kernel/src/__tests__/typed-tables.integration.test.ts`

Interfaces:

- Produces tables (all company-scoped, RLS, row audit): `kernel.entity_type_settings(entity text, requires_approval bool`

`default false, posts bool default false, unique(company_id, entity))`; `kernel.record_snapshots(entity, record_id uuid, status text, payload jsonb, reason text, chain_id uuid, seq)` with immutable trigger (pattern `audit_log_immutable`, 0002); `kernel.state_recipients(entity, status text, emails text[], unique(company_id, entity, status))`; `kernel.mail_log(entity, record_id, recipients text[], template_id, template_version int, status text, provider_id text)`; `kernel.reference_registry` as a SQL function contract `kernel.register_reference(entity text, schema text, table text, column text)` + table `kernel.references`.

- Produces function `kernel.typed_columns(schema text, table text)` that adds `status text not null default 'draft'`, `requires_approval bool`, `posts bool` (documento only), `is_active bool not null default true` (maestro only), `void_reason text`, `deleted_at timestamptz` (maestro only), with a check constraint on the state codes, and a `before insert` trigger copying flags from `entity_type_settings` (Review Focus 4).

- **Step 1: Failing integration tests** — snapshot update/delete raise `record_snapshots_immutable`; company B cannot read company A rows in each table; a table passed to `typed_columns` copies flags at insert and keeps them after the setting changes; invalid status refused.

- **Step 2: Run** `pnpm --filter @erp/kernel test:integration -- typed-tables` → FAIL.
- **Step 3: Write the migration**; `pnpm db:reset`.
- **Step 4: Run** → PASS. **Step 5: Commit** `feat(kernel): typed entity tables`.

Task 4: Settings, recipients and their processes (Parámetros)

Files:

- Create: `packages/kernel/src/typed/settings.processes.ts`
- Modify: `packages/kernel/src/module/kernel-module.ts` (register)
- Test: `packages/kernel/src/typed/__tests__/settings.integration.test.ts`

Interfaces:

- Produces processes: `kernel.entity_type_settings.update({ entity, requiresApproval, posts })` (posts refused for maestro), `kernel.state_recipients.update({ entity, status, emails })` (each

e-mail validated with the existing validator; status must be legal for the type). Both admin-only, audited.

- **Step 1: Failing tests** — non-admin `forbidden`; bad e-mail `invalid_input` naming the address; `posts:true` on a maestro refused; status `posted` on a maestro refused.
- **Step 2: Run** `pnpm --filter @erp/kernel test:integration --settings` → FAIL. **Step 3: Implement. Step 4: Run** → PASS. **Step 5: Commit**.

Task 5: Lifecycle runner, generic processes and snapshots

Files:

- Create: `packages/kernel/src/typed/runner.ts`, `packages/kernel/src/typed/processes.ts`, `packages/kernel/src/typed/snapshot.ts`
- Modify: `packages/kernel/src/module/loader.ts` (register generic processes for each typed entity)
- Test: `packages/kernel/src/typed/__tests__/runner.integration.test.ts` (with a test-only documento + lines entity in a fixture schema)

Interfaces:

- Consumes: Task 2 table, Task 3 tables.
- Produces: `interface TypedHooks { validate?(ctx, record, mode: "save"|"approve"|"post"): Promise<void>; number?(ctx, record): Promise<string>; post?(ctx, record): Promise<void> }` registered by a module through `ModuleDef.typedHooks?: Record<entityId, TypedHooks>`.
- Produces: `runLifecycle(ctx: ProcessContext, entity: ResolvedEntityDef, recordId: string, action: LifecycleAction, input: { data?: unknown; reason?: string; expectedUpdatedAt?: string }): Promise<{ status: RecordStatus; number?: string }>`.
- Produces processes per typed entity: `<id>.save`, `<id>.submit`, `<id>.approve`, `<id>.reject`, `<id>.void`, `<id>.delete`; `reason` required for reject and void.
- Produces: `writeSnapshot(ctx, entity, recordId, status, reason)` — payload = header + lines (`typed.lines`) + `extra`; called in the same transaction for every data change or transition of a documento past `draft`.

- **Step 1: Failing tests** — illegal action → `invalid_state` with from/action; data edit in `pending` → `invalid_state`; `save` from draft numbers via hook when `numbering:"on_save"`; edit in `saved` writes one snapshot with the new data; each transition writes one snapshot; `expectedUpdatedAt` older than the row → `stale_record` (Review Focus 3); `delete` in `saved` refused, in `draft` removes the row and audits it; maestro `delete` sets `deleted_at` and refuses when `kernel.references` finds a row, message hints Activo = no; editing an approval field of an approved maestro with requires_approval moves it to `pending`, editing another field does not (D10); creator approves own record successfully.
- **Step 2: Run** `pnpm --filter @erp/kernel test:integration -- runner` → FAIL. **Step 3: Implement. Step 4: Run** → PASS. **Step 5: Commit** `feat(kernel): lifecycle runner and snapshots`.

Task 6: Automatic posting

Files:

- Modify: `packages/kernel/src/typed/runner.ts`
- Test: `packages/kernel/src/typed/__tests__/autopost.integration.test.ts`

Interfaces:

- Behaviour (D8): when `posts` is true, `save` (no approval) or `approve` (approval) then emits outbox event `<id>.ready_to_post` and calls the module `post` hook in the same transaction (no savepoint); success → status `posted` + snapshot; any posting error propagates, `runProcess` rolls back the whole action (data, number, snapshots, outbox) and the caller gets the posting error code; the document keeps its previous state (`draft` on save, `pending` on approve). For entities whose posting is themselves (accounting entries), the `post` hook is the accounting post.
- **Step 1: Failing tests** — posts on save; posts on approve; hook throwing `period_closed` on save → process fails with `period_closed`, row unchanged in `draft` (edits not persisted), no number, no snapshot, no outbox row (Review Focus 1); same on approve → stays `pending`; after reopening the period, approve posts.
- **Step 2: Run** `… -- autopost` → FAIL. **Step 3: Implement. Step 4: Run** → PASS. **Step 5: Commit**.

Task 7: Maestro usability gate on documents

Files:

- Create: `supabase/migrations/0041_kernel_record_usable.sql` (`kernel.record_usable(entity text, id uuid) returns bool`, security definer, reads status/is_active/deleted_at/flags)
- Modify: `packages/kernel/src/typed/runner.ts` (on save/approve/post, every `ref` field of the documento and its lines pointing to a maestro must be usable and usable by the caller (Task 9 `use` right))
- Test: `packages/kernel/src/typed/__tests__/usable.integration.test.ts`
- **Step 1: Failing tests** — documento referencing a `pending` maestro refused at save with `maestro_not_usable` naming the field and record; inactive maestro refused; maestro `saved` without approval accepted; check reruns at approve.
- **Step 2–4:** run `… -- usable` FAIL → implement → PASS. **Step 5: Commit**.

Task 8: Migrate core masters and accounting entries

Files:

- Create: `supabase/migrations/0042_core_masters_typed.sql`, `supabase/migrations/0043_accounting_entries_typed.sql`
- Modify: `packages/modules/core/src/entities.ts`, `packages/modules/core/src/processes/*`, `packages/modules/accounting/src/entities.ts`, `packages/modules/accounting/src/processes/entry.ts`, `entry-post.ts`, `entry-return.ts`, `packages/modules/accounting/src/index.ts` (typedHooks)
- Test: existing suites + `packages/modules/core/src/__tests__/typed-masters.integration.test.ts`, `packages/modules/accounting/src/__tests__/entry-lifecycle.integration.test.ts`

Interfaces:

- Core masters: `status active/inactive` → `is_active` (active→true) and lifecycle `status='saved'` (or `approved`); views keep a computed `status` column for one release for compatibility; `invoice_ready` additionally requires `kernel.record_usable` (update `core.v_party_readiness` in 0042). Register references for every FK to a master.
- Accounting entries: `draft` → `draft`, `pending` → `pending`, `posted` / `reversed` → `posted` with `extra.reversed=true` mark kept as today's `reversed_by` link, `voided` → `voided`; `requires_approval` from

`require_approval_on_manual` for manual journals; `posts=true`; hooks: `validate = validateLines`, `number = accounting.next_entry_number` at post, `post` = current post logic. `accounting.entry.return_to_draft` replaced by `reject` (D2); the four-eyes setting stays as an optional accounting check inside its `validate(mode:"approve")`, default off.

- **Step 1: Failing tests** — a migrated party is usable; an inactive one is not; an entry goes draft→saved→posted automatically without approval; with approval draft→saved→pending→approved→posted; reject → `rejected`, edit, save, resubmit.
- **Step 2: Run** `pnpm --filter @erp/module-core test:integration` and `pnpm --filter @erp/module-accounting test:integration` → new tests FAIL.
- **Step 3: Implement**; update the existing tests that asserted `return_to_draft`/`status:'active'` to the new states without weakening their assertions.
- **Step 4: Run** both suites → all PASS (the 156 accounting tests included).
  Step 5: Commit.

Task 9: Record-level access

Files:

- Create: `supabase/migrations/0044_kernel_record_access.sql`, `packages/kernel/src/typed/access.processes.ts`
- Modify: `kernel.enable_company_rls()` (new version in 0044) to add `kernel.can_access_record(entity, id, 'read')` to typed tables and their line tables; `packages/kernel/src/process/run.ts` guard hook `assertRecordRight(ctx, entity, id, right)`
- Test: `packages/kernel/src/__tests__/record-access.integration.test.ts`

Interfaces:

- Table `kernel.record_access(entity, record_id, grantee_role_id uuid null, grantee_user_id uuid null, can_read, can_use, can_modify, can_delete, can_void, check (exactly one grantee))`.
- `kernel.can_access_record(entity text, id uuid, right text) returns bool`: true if caller is company admin, or no rows exist for the record, or a row for the caller's user or one of their roles grants it; never true if the role lacks the base permission.
- Processes: `kernel.record_access.set({ entity, recordId, grants: Grant[] })` (replaces the set, audited), admin-only. Rights allowed: maestro

read/use/modify/delete; documento read/modify/void.

- **Step 1: Failing tests** — restricted user cannot select the maestro, its lines, nor see it in `v_*` views; admin can; granted user can read but `use` false → documento save refused `record_forbidden`; aggregate report (trial balance) totals identical for restricted user and admin (D11, Review Focus 2); `void` right enforced on documento.

- **Step 2–4:** `pnpm --filter @erp/kernel test:integration -- record-access` FAIL → implement → PASS; rerun accounting suite. **Step 5: Commit**.

Task 10: Module menu sections and menu access

Files:

- Modify: `packages/kernel/src/registry/types.ts` (`NavItem.section?: "maestros"|"documentos"|"reportes"|"parametros"`), `packages/kernel/src/module/loader.ts` (validator), module manifests `packages/modules/{core,accounting,support}/src/index.ts`, shell nav builder in `packages/ui/src/shell/`
- Create: `supabase/migrations/0045_kernel_menu_access.sql`, `packages/kernel/src/typed/menu-access.processes.ts`
- Test: `packages/kernel/src/__tests__/module-loader.test.ts`, `packages/kernel/src/__tests__/menu-access.integration.test.ts`, `packages/ui/src/shell/__tests__/AppShell.test.tsx`

Interfaces:

- Every module's level-1 nav contains groups with ids `<module>.maestros|documentos|reportes|parametros` (labels Maestros, Documentos, Reportes, Parámetros); loader throws `module x: missing menu section <s>`. Typed entities' list pages auto-placed in their section; their list reports in Reportes; `entity_type_settings`, templates, recipients in Parámetros. Empty section renders an empty state ("Aún no hay elementos en esta sección") instead of being dropped.

- Table `kernel.menu_access(nav_key, grantee_role_id, grantee_user_id, allowed bool)`; function `kernel.menu_allowed(nav_key) returns bool` (admins always true; no rows → true); shell filters; route guard in `apps/web/src/proxy.ts` (or the existing layout guard) and `runProcess` refuse processes listed under a hidden item with `forbidden`.

- **Step 1: Failing tests** — loader rejects a module missing a section; shell hides a denied item for the user, shows it for admin; direct URL to a denied

page returns the forbidden screen; the item's process refused.

- **Step 2–4:** run the three test files FAIL → implement → PASS. **Step 5: Commit**.

Task 11: PDF templates (data, rendering, defaults)

Files:

- Create: `supabase/migrations/0046_kernel_doc_templates.sql`, `packages/kernel/src/templates/{schema.ts,variables.ts,render.tsx,processes.ts,defaults.ts}`
- Test: `packages/kernel/src/templates/__tests__/render.test.ts`, `…/templates.integration.test.ts`

Interfaces:

- Table `kernel.doc_templates(entity, scope 'record'|'list', name, background_file_id uuid null (kernel files), blocks jsonb, is_default bool, version int)`; one default per (company, entity, scope).
- `type Block = { kind: "text"|"variable"|"field"|"lines"|"image"|"page_number"; x: number; y: number; w: number; h?: number; font?: { size: number; bold?: boolean }; align?: "left"|"center"|"right"; value?: string; columns?: string[] }` (zod in `schema.ts`).
- `resolveVariables(ctx, entity, record | rows): Record<string, string>` — `empresa.nombre|nit|direccion|ciudad|telefono|logo`, `usuario.nombre`, `fecha_impresion`, record fields formatted with existing `exports/format.ts` (money `$ 1.234.567,00`), masked fields → "".
- `renderTemplatePdf(template, vars, rows?, background?): Promise<Buffer>` with `@react-pdf/renderer`; background PDF page rasterised once per version as page image.
- `defaultTemplate(entity, scope): Template` generated from entity fields when the company has none.
- Processes: `kernel.doc_template.save` (new version each save), `kernel.doc_template.set_default`, `kernel.doc_template.print({ entity, recordId? , gridState?, templateId? }) → { fileId }` (list scope uses the grid state and access rules).
- **Step 1: Failing tests** — variables resolve with Café del Norte data (`NIT 900373115-3`); masked field prints empty; money formatting; default template renders every visible field; list print excludes rows hidden by record access; save bumps version.

- **Step 2–4:** FAIL → implement → PASS. **Step 5: Commit**.

Task 12: State e-mails and Notificar

Files:

- Modify: `packages/infra/src/mailer.ts` (Resend transport), `packages/kernel/src/typed/runner.ts` (emit `<id>.status_changed` outbox event)
- Create: `packages/kernel/src/typed/notify.ts` (outbox consumer + `kernel.record.notify` process)
- Test: `packages/kernel/src/typed/__tests__/notify.integration.test.ts`, `packages/infra/src/__tests__/mailer.test.ts`

Interfaces:

- Consumer: on `status_changed`, read `state_recipients` for (entity, new status); if any, render the default record template (Task 11) and send one mail with the PDF attached; write `mail_log`; idempotent per event id.
- `kernel.record.notify({ entity, recordId, to: string[], subject, message })` — validates addresses, sends with PDF, logs. Copy: subject default `"{tipo} {numero} · {estado}"`.
- **Step 1: Failing tests** — reaching `approved` with a recipient list sends one mail to the Docker SMTP sink with a PDF attachment; redelivery sends none extra; no list → no mail; Notificar to an invalid address refused; Resend transport posts the expected payload (mocked fetch).
- **Step 2–4:** FAIL → implement → PASS. **Step 5: Commit**.

Task 13: Import and export for typed entities

Files:

- Modify: `packages/kernel/src/import/{entity.ts,run.ts,validate.ts}`, `packages/kernel/src/exports/entity.ts`
- Test: `packages/kernel/src/__tests__/import.test.ts`, `packages/kernel/src/import/__tests__/typed-import.integration.test.ts`, `packages/kernel/src/__tests__/exports.test.ts`

Interfaces:

- Import of documentos: a header sheet + lines sheet joined by a file key column `ref`; each document created through `<id>.create` then lines, landing `draft`; maestros through create then `save` (→ `saved`, then `submit` → `pending` if requires_approval). Never approve or post.
- Export: status column with es-419 labels; rows filtered by record access.
- **Step 1: Failing tests** — imported documento lands `draft`; imported maestro needing approval lands `pending` (Review Focus 5); a restricted maestro is not matched by import key for a narrowed user; export omits restricted rows and shows "Grabado".
- **Step 2–4:** FAIL → implement → PASS. **Step 5: Commit**.

Task 14: Prototype for the action bar, Versiones, Acceso and template editor

Files:

- Create: `docs/superpowers/specs/2026-11-02-entity-types-prototype.html`
- **Step 1:** Build a static prototype on the shell v5 of `2026-10-27-erp-ia-phase1-prototype.html`: maestro record with Grabar · Eliminar · Nuevo · Imprimir; documento in each state with the buttons the Task 2 `allowedActions` table yields; state chip; Versiones tab (list + compare two); Acceso tab; Notificar dialog; template editor (block list left, live preview right, background upload); module menu with the four sections.
- **Step 2:** Owner reviews; record changes in the design before Task 15. (No code until approved.)

Task 15: Standard record action bar and tabs

Files:

- Create: `packages/ui/src/record/RecordActions.tsx`, `packages/ui/src/record/VersionsTab.tsx`, `packages/ui/src/record/AccessTab.tsx`, `packages/ui/src/record/NotifyDialog.tsx`
- Modify: `packages/ui/src/record/RecordFrame.tsx`, `apps/web/src/app/(app)/r/[entity]/[id]/{record-frame.tsx,record-form.tsx}`, `apps/web/src/lib/records/tab-actions.ts`, `apps/web/src/app/(app)/accounting/entries/` (use RecordActions; remove `EntryStatusBar` buttons), `packages/i18n/messages/{es-419,en}.json`

- Test: `packages/ui/src/record/__tests__/RecordActions.test.tsx`, `apps/web/e2e/typed-entities.spec.ts`

Interfaces:

- `RecordActions({ entityType, status, flags, rights: { modify; delete; void; approve }, dirty, onAction(action: LifecycleAction | "new" | "print" | "notify") })` — shows exactly `allowedActions` ∩ rights; Eliminar only when Task 2 allows; Aprobar/No Aprobar only with approve permission; a refused action (including a posting refusal) shown as an inline alert under the bar with the reason, the form keeping the unsaved edits. Datos adicionales tab uses the existing `custom-fields` record tab for every typed entity.

- **Step 1: Failing tests** — unit: buttons per (type, status, flags, rights) matrix; e2e: create a party → Grabar → Imprimir downloads a PDF; a comprobante saved without approval shows Contabilizado; with approval, Aprobar by the same user posts; Anular asks a reason; Versiones lists snapshots.

- **Step 2: Run** `pnpm vitest run packages/ui/src/record` and `pnpm --filter web e2e -- typed-entities` → FAIL.
- **Step 3: Implement** per the approved prototype. **Step 4: Run** → PASS; update affected screenshots (`apps/web/e2e/__screenshots__`) only for intended changes. **Step 5: Commit**.

Task 16: Parámetros screens and list-report print

Files:

- Create: `apps/web/src/app/(app)/[module]/parametros/` pages for entity type settings, state recipients, templates (editor from the prototype), menu access (admin), list "Imprimir" on grids in `packages/ui/src/grid/DataGrid.tsx`

- Test: `apps/web/e2e/typed-parametros.spec.ts`

- **Step 1: Failing e2e** — admin turns on requires approval for parties → a new party lands Pendiente; adds a recipient for Aprobado → approval sends a mail (SMTP sink); edits a template's fixed text → Imprimir shows it; grid Imprimir downloads the list PDF; non-admin cannot open Parámetros.

- **Step 2–4:** FAIL → implement → PASS. **Step 5: Commit**.

Task 17: Assistant, user guides, rules and close

Files:

- Modify: assistant tool registry for typed processes (registry only; writes via confirmation cards), `docs/user-guide/es-419/` (new `maestros-y-documentos.md`, `plantillas-pdf.md`, `notificaciones.md`, `acceso-por-registro.md`; update `terceros.md`, `productos.md`, `contabilidad/` comprobantes, `importar.md`, `exportar.md`, `navegacion.md`), `docs/roadmap.md` (2A.5 done)
- Test: `pnpm evals` structural evals for approve/void cards.
- **Step 1:** Evals: assistant asks confirmation before `approve` / `void`; refuses a document it cannot `modify`.
- **Step 2:** Guides with screenshots (`pnpm --filter web guide:shots`).
- **Step 3: Full gate** — `pnpm lint && pnpm typecheck && pnpm check:rules && pnpm test && pnpm test:integration && pnpm --filter web e2e && pnpm build` → all green; CI green on push (owner's word).
- **Step 4: Commit** `docs: phase 2A.5 guides and roadmap`.

Order and parallelism

1 → 2 → 3 → 4/5 → 6 → 7 → 8 → 9 → 10; 11 and 14 may start after 3; 12 after 11; 13 after 9; 15 after 8, 9, 14; 16 after 11, 12, 15; 17 last.